Data Analytics II

Loading the reequired libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Importing the dataset

```
data = pd.read_csv('Social_Network_Ads.csv')
```

```
data
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

```
data.head(5)
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
data.tail()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

```
data.shape
```

```
(400, 5)
```

```
data.columns
```

```
Index(['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased'],
dtype='object')
```

```
data.describe()
```

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

```
data.isnull().sum()
```

	0
User ID	0
Gender	0
Age	0
EstimatedSalary	0
Purchased	0

dtype: int64

```
# requier 2 columns age and salary from given data frame
data.iloc[:,2:4]
```

	Age	EstimatedSalary
0	19	19000
1	35	20000
2	26	43000
3	27	57000
4	19	76000
...	...	...
395	46	41000
396	51	23000
397	50	20000
398	36	33000
399	49	36000

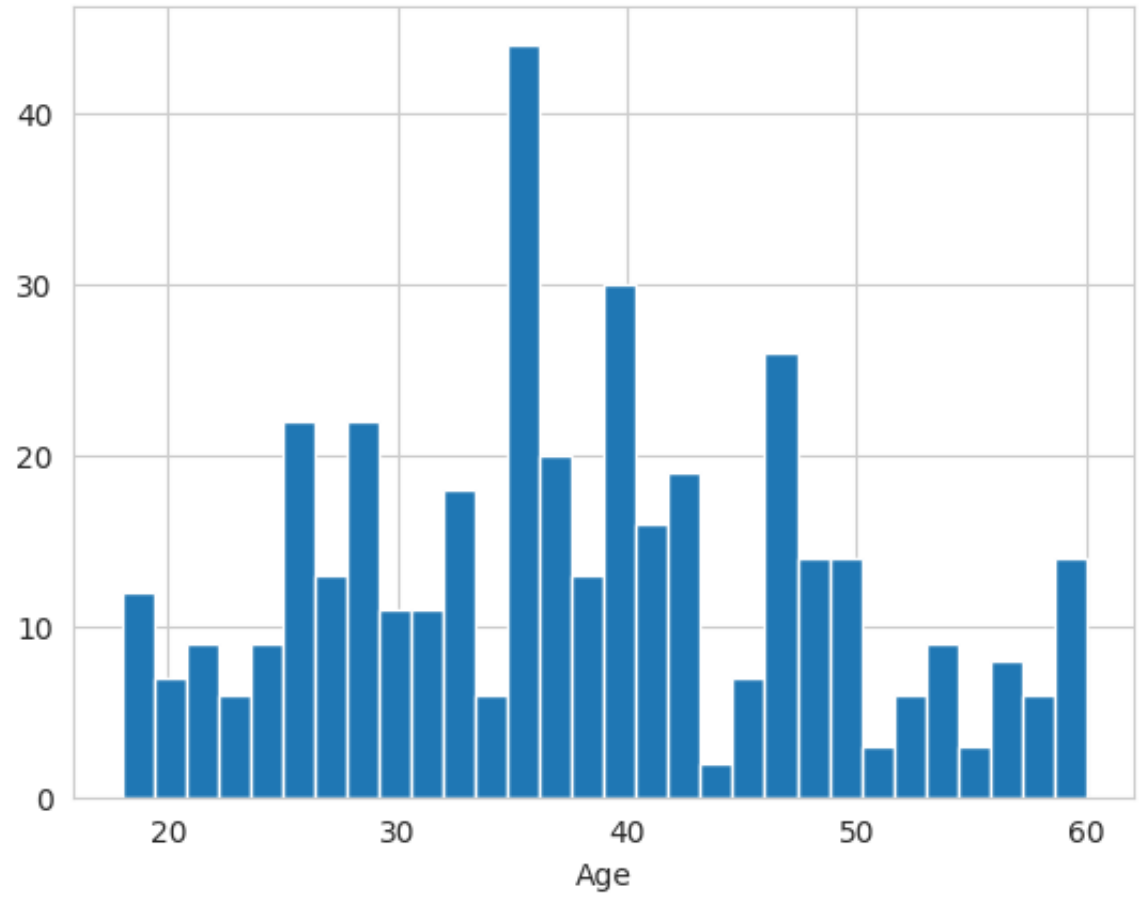
400 rows × 2 columns

```
# in the form of numpy array
ndata= data.iloc[:,2:4].values
```

```
import seaborn as sns
```

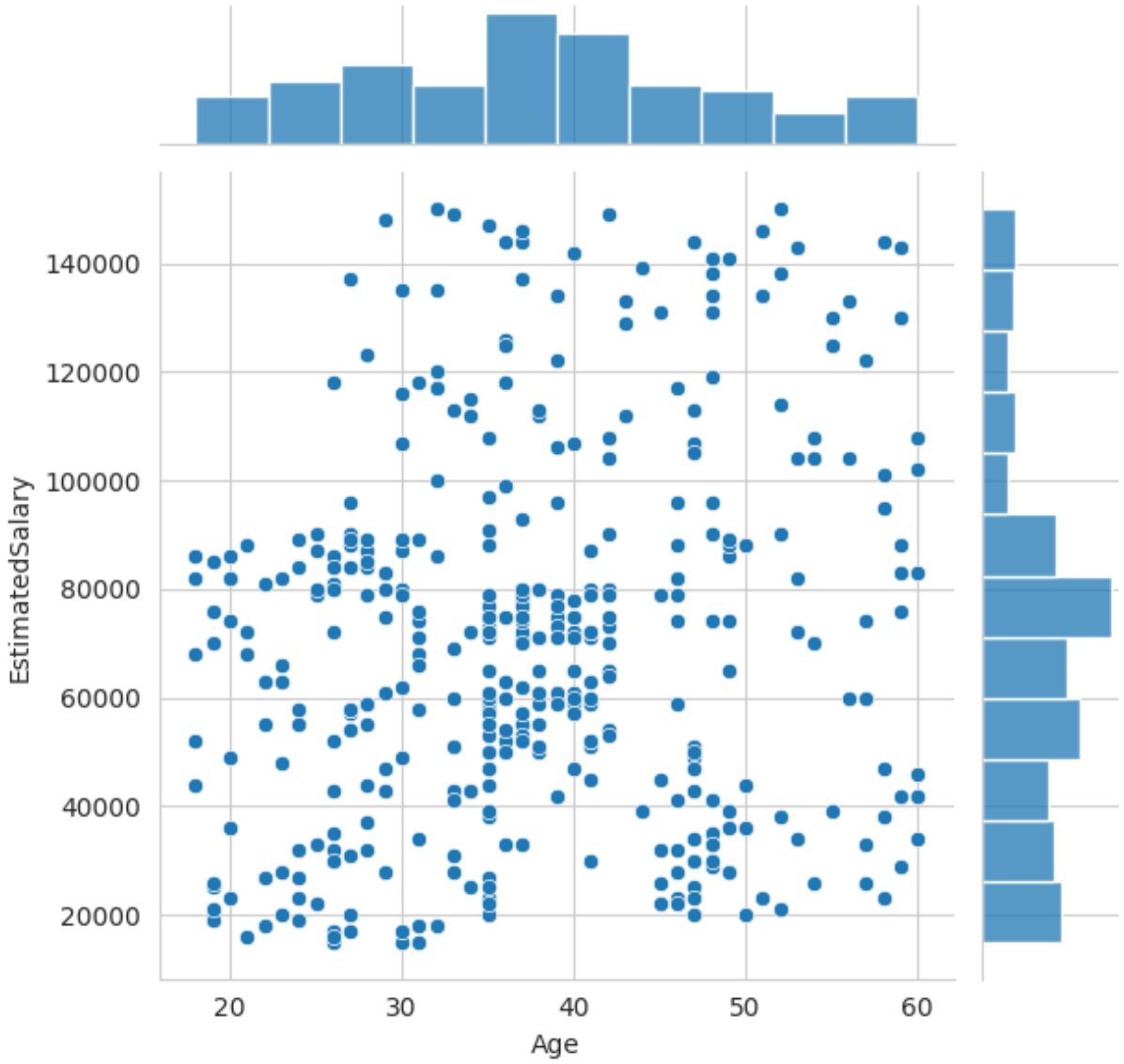
```
sns.set_style('whitegrid')
data['Age'].hist(bins=30)
plt.xlabel('Age')
```

```
Text(0.5, 0, 'Age')
```



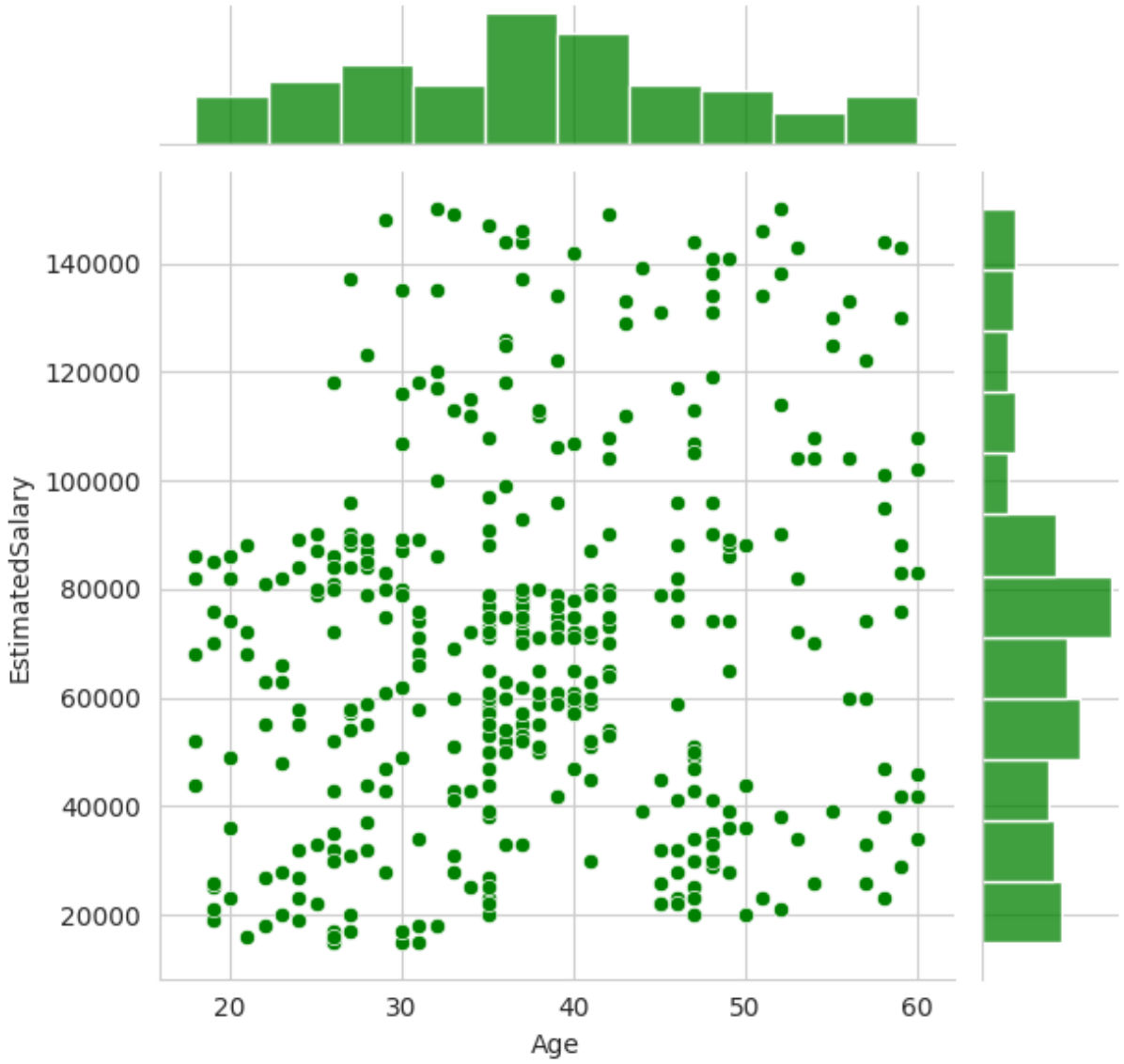
```
sns.jointplot(x='Age', y='EstimatedSalary', data = data)
```

```
<seaborn.axisgrid.JointGrid at 0x781d8e0fc5f0>
```



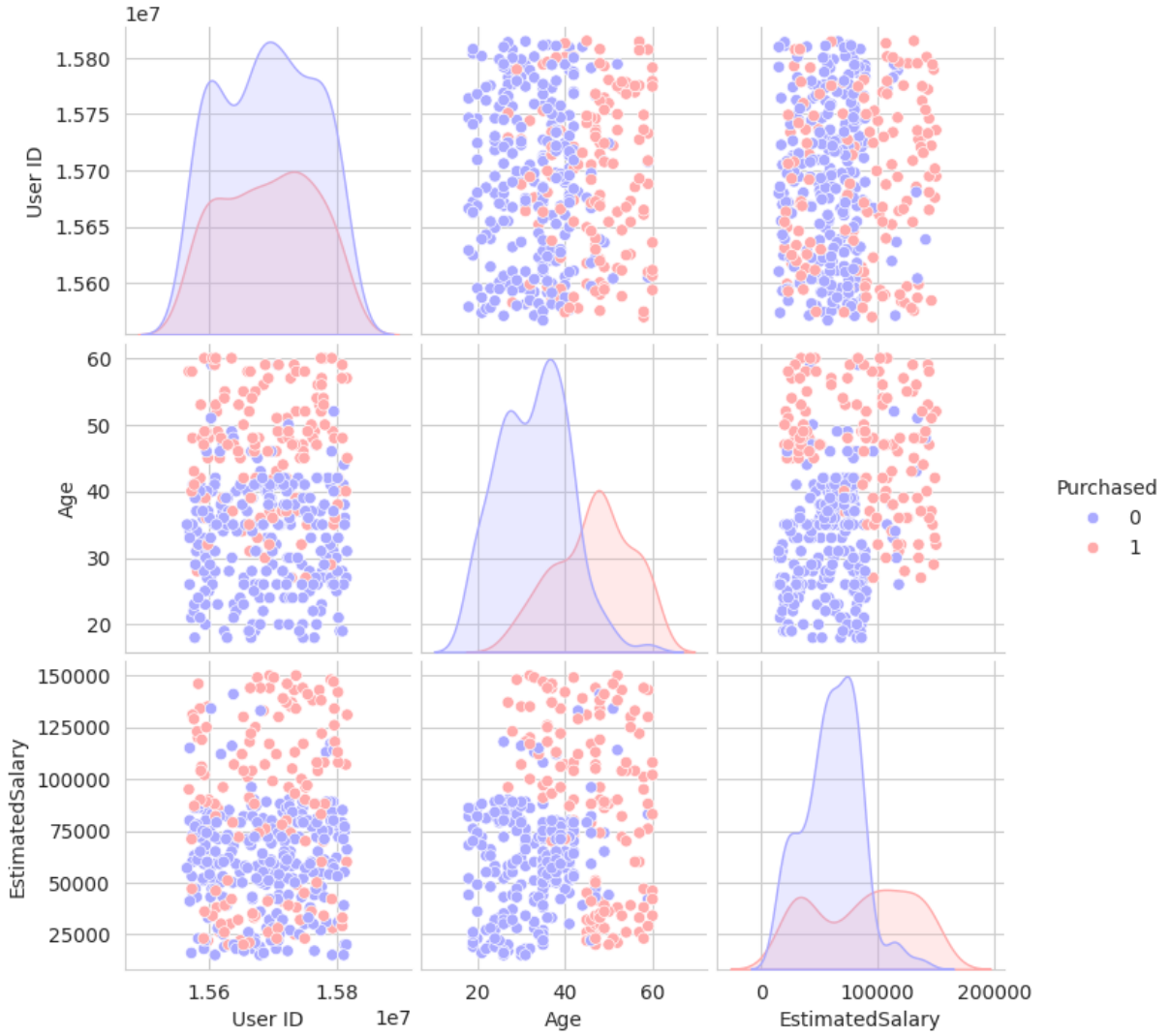
```
sns.jointplot(x='Age',y='EstimatedSalary',data= data,color='green')
```

```
<seaborn.axisgrid.JointGrid at 0x781d81b18a10>
```



```
sns.pairplot(data,hue='Purchased',palette='bwr')
```

```
<seaborn.axisgrid.PairGrid at 0x781d951e8140>
```



Logistic Regression

```
# ** Split the data into training setand testing set using train_te

from sklearn.model_selection import train_test_split
```

```
X =data[['Age','EstimatedSalary']]
y = data['Purchased']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
```

```
X_train.shape
```

```
(300, 2)
```

```
X_test.shape
```

```
(100, 2)
```

```
y_train.shape
```

```
(300,)
```

```
y_test.shape
```

```
(100,)
```

Train and fit a logistic-regression model on the training set.

```
from sklearn.linear_model import LogisticRegression
```

```
logmodel=LogisticRegression()
logmodel.fit(X_train,y_train)
```

▾ LogisticRegression

```
LogisticRegression()
```

Predictions and Evaluations

```
predictions= logmodel.predict(X_test)
```

Create a classification report for the model.

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

           0       0.89      0.96      0.92        68
           1       0.89      0.75      0.81        32

    accuracy                           0.89       100
   macro avg       0.89      0.85      0.87       100
weighted avg       0.89      0.89      0.89       100
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, predictions)
print(cm)
```

```
[[65  3]
 [ 8 24]]
```

This Confusion Matrix tells us that there were 68 correct predictions and 32 incorrect ones,meaning the model overall accomplished an 68% accuracy rating.